מנגנון טיפול בשגיאות בקריאות מערכת בלינוקס

כאשר תהליך במרחב המשתמש (User Space) מבצע **קריאת מערכת (System Call)**, הוא למעשה מבקש שירות ישירות מ**ליבת מערכת ההפעלה (Kernel)**. שירותים אלה, כגון פתיחת קובץ, כתיבה לדיסק, או יצירת תהליך, עשויים להיכשל מסיבות מגוונות (למשל, קובץ לא קיים, חוסר הרשאות, חוסר זיכרון, או פרמטרים שגויים).

1. הצורך במנגנון דיווח שגיאות

קריאת מערכת היא גבול מעבר קריטי בין **מרחב המשתמש** ל**מרחב הליבה**. הליבה פועלת ברמת הרשאה גבוהה יותר (Mode Kernel) ויש לה אחריות לאכוף את כללי המערכת. כאשר קריאת מערכת נכשלת, יש **הכרח מוחלט** במנגנון שדרכו הליבה תוכל:

א. לדווח על הכישלון: להודיע לתהליך הקורא כי הבקשה לא בוצעה בהצלחה.

ב. לספק את הסיבה הספציפית לכישלון: לאפשר לתהליך לדעת מדוע הבקשה נכשלה, כדי שיוכל לטפל בשגיאה בצורה נאותה (למשל, לנסות שוב, לדווח למשתמש, או לסיים את פעולתו).

בלינוקס ובמערכות תואמות POSIX, **קריאות מערכת** שעלולות להיכשל בדרך כלל מחזירות ערך מוסכם המציין הצלחה או כישלון. בדרך כלל:

- **הצלחה:** מוחזר ערך **אי-שלילי** (לרוב 0 =<), המייצג את התוצאה החוקית של הקריאה (למשל, מספר בייטים שנכתבו, מזהה תהליך חדש, או file descriptor).
- **כישלון:** מוחזר ערך **שלילי** מוסכם, לרוב -1 ערך זה הוא האות הראשוני והחשוב ביותר המעיד על כשל.

כאשר קריאת מערכת מחזירה **1-** הסיבה הספציפית לכישלון אינה מועברת דרך ערך ההחזרה עצמו, אלא דרך מנגנון נפרד: **המשתנה הגלובלי** errno.

2. המשתנה הגלובלי errno

המשתנה **errno** (קיצור של "Error Number") הוא המנגנון הסטנדרטי והמרכזי ביותר לדיווח על שגיאות הנובעות מקריאות מערכת וקריאות לספריות (Library Calls) בסביבת POSIX.

© יצחק נודלר

- **תפקיד:** כאשר קריאת מערכת נכשלת (ומחזירה 1-), הליבה מעדכנת את הערך של errno למספר שלם **חיובי** המייצג את **קוד השגיאה הספציפי** שהתרחשה.
- **טווח:** כל קוד שגיאה מייצג מצב כישלון מוגדר (לדוגמה, 2 = "אין קובץ או ספרייה כזו", 13 = "הרשאה נדחתה").

3. היכן מוגדר המשתנה errno?

מבחינה טכנית, **errno אינו משתנה גלובלי רגיל**. הוא ממומש בצורה שמאפשרת לו להיות **פר-תהליכי (Per-Process)** או **פר-תהליכון (Per-Thread)**.

- במערכות תואמות POSIX, ובפרט בלינוקס, errno מוגדר למעשה **כמאקרו** (Macro) המתרחב לביטוי שמחזיר את הערך המאוחסן במיקום זיכרון השייך לתהליך (או לתהליכון) הנוכחי.
- הסיבה למימוש זה היא **בטיחות תהליכונים (Thread Safety)**. אם errno היה משתנה גלובלי רגיל, שני תהליכונים שונים באותו תהליך המבצעים קריאות מערכת בו-זמנית היו דורסים זה את ערכו של זה, מה שמוביל למצב מירוץ (Race Condition) ולמידע שגיאה שגוי. הגישה הפר-תהליכון מבטיחה שכל תהליכון יכול לקרוא את קוד השגיאה שלו באופן אמין.

4. קובץ ההגדרה הדרוש

יש הכרח לבצע `include` של קובץ הגדרה ספציפי כדי להשתמש ב-errno:

```
#include <errno.h>
```

קובץ כותר זה מגדיר את המאקרו של errno ואת רשימת קודי השגיאה המוסכמים כמאקרואים (למשל, `EACCES` עבור "Access Denied").

הערה חשובה: לאחר קריאת מערכת כושלת, תוכנית היישום **חייבת** לקרוא את הערך של errno **מייד** לאחר הכישלון, מכיוון שכל קריאת ספרייה או קריאת מערכת עוקבת (אפילו קריאה מוצלחת) עלולה **לדרוס** את ערכו של errno.

© יצחק נודלר

5. טבלת השגיאות האפשריות (קודי שגיאה)

קודי השגיאה הם מספרים שלמים חיוביים (כפי שצוין, הערך של errno). לכל מספר כזה מוגדר שם סמלי (Symbolic Name) המתחיל באות **'E'** ומשמש לקריאות קוד קלה יותר.

קוד מספרי (ב-errno)	שם סמלי (מאקרו)	תיאור קצר
1	EPERM	Operation not permitted (אין הרשאה)
2	ENOENT	No such file or directory (קובץ לא נמצא)
4	EINTR	Interrupted system call (נקטעה על ידי אות)
11	EAGAIN / EWOULDBLOCK	Resource temporarily unavailable (נסה שוב)
13	EACCES	Permission denied (הרשאה נדחתה)
22	EINVAL	Invalid argument (ארגומנט לא חוקי)

היכן מוגדרת הטבלה?

- **ההגדרות הסמליות** (כגון ENOENT, EACCES וכו') מוגדרות בקובץ הכותר **<errno.h>**.
- **ההגדרות המספריות בפועל** של הקודים המספריים מוגדרות ברמת הליבה, וקובץ הכותר של הליבה (asm-generic/errno-base.h וכו') מועתק לספריית הכותרות של המשתמש בעת בניית המערכת.

© יצחק נודלר

6. קובץ ההגדרה הדרוש לקודי השגיאה

יש הכרח לכלול את קובץ הכותר `<errno.h>` כדי להשתמש בשמות הסמליים של קודי השגיאה (לדוגמה, כדי לכתוב `if (errno == EACCES)`).

7. הפונקציה `perror()`

בעוד ש- errno מספק את הקוד המספרי והסמלי של השגיאה, לעיתים קרובות אנו רוצים להציג למשתמש הודעת שגיאה קריאה בשפה טבעית. זהו תפקידה של הפונקציה `perror()`.

מה הפונקציה עושה?

הפונקציה perror() מקבלת מחרוזת קלט (String) ומדפיסה אותה לפלט השגיאה הסטנדרטי (stderr), ולאחריה היא מוסיפה נקודתיים, רווח, ואת תיאור השגיאה המילולי המתאים לערך הנוכחי של errno. לבסוף היא מוסיפה תו שורה חדשה (\n)

איך יש לקרוא לה?

יש לקרוא ל-perror() באופן הבא:

```
void perror(const char *s);
```

כאשר `s` היא המחרוזת המותאמת אישית של המתכנת, המעידה על הקטע בתוכנית שבו הכישלון אירע.

דוגמה לשימוש קלאסי:

```
if (open("non_existent_file.txt", O_RDONLY) == -1) {
    perror("Failed to open file");
    // ... טיפול בשגיאה ...
```


© יצחק נודלר

```
}
```

אם קריאת המערכת open נכשלה כי הקובץ לא נמצא (errno = 2 = ENOENT), הפלט יהיה:

```
Failed to open file: No such file or directory
```

8. קובץ ההגדרה הדרוש ל- perror()

יש הכרח לכלול את קובץ הכותר הבא כדי להשתמש ב-perror():

```
#include <stdio.h>
```

(מכיוון ש-perror היא פונקציה סטנדרטית של ספריית הקלט/פלט).

היבטים נוספים

I. הפונקציה strerror()

לצד perror(), קיימת גם הפונקציה strerror(), המוגדרת ב**<string.h>**:

```
char *strerror(int errnum);
```

פונקציה זו **מחזירה מחרוזת (string)** המכילה את תיאור השגיאה המילולי עבור קוד השגיאה המספרי errnum. היא שימושית כאשר רוצים לשלב את תיאור השגיאה ביומן (log) או בהודעת שגיאה מותאמת אישית שאינה מודפסת ישירות ל-stderr.

II. בדיקת ערך ההחזרה: עקרון יסוד

כלל אצבע בתכנות מערכות הוא: **יש לבדוק את ערך ההחזרה של כל קריאת מערכת וכל קריאת ספרייה שעלולה להיכשל באופן עקבי**. זוהי הדרך היחידה לוודא הצלחה או לזהות כישלון.

© יצחק נודלר

מבנה בדיקת שגיאות טיפוסי:

```
ssize_t bytes_written;
// ...
bytes_written = write(fd, buffer, count);
if (bytes_written == -1) {
    // 1. קריאת המערכת נכשלה
    // 2. ה-kernel עדכן את ערך errno
    perror("Error writing to file descriptor"); //  הדפסת הודעת
שגיאה קריאה
    exit(EXIT_FAILURE);
}
// 3. ה, אם הגענו לכאן-write חלקית (ייתכן שרק הצליח)
```

דוגמא קוד להמחשת טיפול בשגיאות System Calls

זיהוי כישלון ודיווח כללי

דוגמה זו מדגימה את השימוש הבסיסי והנפוץ ביותר במנגנון: **בדיקת ערך החזרה של 1-** ושימוש בפונקציה `perror()` לדיווח מיידי וידידותי למשתמש.

תרחיש: ניסיון פתיחת קובץ שלא קיים, או שאין למשתמש הרשאה לפתוח אותו.

```
#include <stdio.h>      // עבור perror
#include <stdlib.h>
#include <unistd.h>     // עבור קריאת המערכת close
#include <fcntl.h>      // עבור קריאת המערכת open ודגלי גישה
(O_RDONLY)

// הפונקציה המרכזית
int main() {
    int fd;
    const char *filename = "private_file.txt"; //  קובץ שאינו קיים
או נעול

    printf("Attempting to open file: %s\n", filename);
```


© יצחק נודלר

```
    // קריאת המערכת open(2)
    // מחזירה file descriptor (ערך >= 0)  1- או ,במקרה של הצלחה
במקרה של כישלון
    fd = open(filename, O_RDONLY);

    // בדיקת ערך ההחזרה של קריאת המערכת
    if (fd == -1) {
        // הכישלון אושר. הליבה עדכנה את errno.

        // ב שימוש-perror() לדיווח.
        // פונקציה זו משתמשת בערך הנוכחי של errno ומדפיסה:
        // "Failed to open file: [התיאור המילולי של קוד השגיאה]"
        perror("Failed to open file");

        // יציאה עם קוד כישלון
        return EXIT_FAILURE;
    }

    // קוד זה יתבצע רק במקרה של הצלחה
    printf("File opened successfully with file descriptor: %d\n",
fd);
    close(fd);

    return EXIT_SUCCESS;
}
```


© יצחק נודלר